

Московский физико-технический институт
Физтех-школа прикладной математики и информатики

СЛОЖНОСТЬ ВЫЧИСЛЕНИЙ
IV СЕМЕСТР

Лектор: *Мусатов Даниил Владимирович*



Автор: *Киселев Николай*
Репозиторий на Github

весна 2026

Содержание

1	Класс P	2
1.1	Базовые определения	2
1.2	Неконструктивные оценки P	2
1.3	Другие классы задач	3
1.4	Асимптотики различных задач	3
2	Класс NP	3
2.1	Определение через сертификат	4
2.2	Некоторые следствия из определений	5
2.3	Задача о рюкзаке и её NP -полнота	10
3	Классы задач, связанные с задачами распознавания	11
3.1	Задача поиска	11
3.2	Задача подсчета	12
3.3	Задача аппроксимации	12
4	Полиномиальная иерархия	12
4.1	Примеры	13
4.2	Формально	14
4.3	Вопрос о коллапсе иерархии	14
4.4	Полнота в полиномиальной иерархии	15
4.5	Альтернирующие Машины Тьюринга	16
5	Пространственная сложность	17
5.1	Задачи из $PSPACE$	19
5.2	Задача $TQBF$	19
5.3	Игра GG и её $PSPACE$ -полнота	20
6	Классы L, NL	22
6.1	$L \stackrel{?}{=} NL$	22
6.2	Сертификатное определение NL	24
6.3	NL -полные задачи	25
7	Схемная сложность	25
7.1	Сложность по размеру	25
7.2	Сложность по времени	27
7.3	Схемы малой глубины для $+$, $\cdot$	28

7.4	Быстрые сумматоры	28
7.5	Быстрые мультипликаторы	28
8	Вероятностная сложность	28
8.1	Вероятностные классы	29
8.2	Метод условных матожиданий	30
9	Интерактивные доказательства	30

1 Класс P

1.1 Базовые определения

Будем рассматривать задачи на распознавание, т.е. дан $A \subset \{0,1\}^*$ и требуется по $x \in \{0,1\}^* \mapsto \begin{cases} 1, x \in A \\ 0, x \notin A \end{cases}$. Пусть M решает данную задачу, т.е. $\forall x (x \in A \Leftrightarrow M(x) = 1)$.

Определение 1.1. $time_M(x)$ — число шагов $M(x)$ при вычислении ответа.

Определение 1.2. $time_M(n) = \max_{x:|x|=n} time_M(x)$

Определение 1.3. $time_M(n) = O(f(n))$, если $\exists C : \forall n : time_M(n) \leq C \cdot f(n)$

Возникает вопрос: можно ли сказать, что $time_A(n) = \min_{M:M \text{ решает } A} time_M(n)$? Нет, но показать это достаточно сложно (Теорема Блюма).

Поэтому мы приходим к данному определению:

Определение 1.4. $\mathbf{DTIME}(t(n)) = \{A | \exists M : M(x) = 1 \Leftrightarrow x \in A, time_M(n) = O(t(n))\}$

Заметим, что для определения **DTIME**, необходимо задать модель вычислений. Обычно такой моделью выбирают многоленточную машину Тьюринга.

Определение 1.5. $\mathbf{P} = \mathbf{DTIME}(poly(n)) = \bigcup_{k=1}^{\infty} \mathbf{DTIME}(n^k)$

Тезис Черча-Тьюринга в сильной форме: Любая задача, эффективно решаемая физическим устройством, решается за полиномиальное время на машине тьюринга.

Пример (Нетривиальные примеры задач из **P**). 1. $\mathbb{P}$ — множество простых чисел

2. Линейное программирование — как пример, нахождения максимума функции на многограннике. Эта задача не бинарная, но вот задача "достижима ли это число на многограннике" принадлежит классу **P**.

3. Симплекс-метод — алгоритм решения

1.2 Неконструктивные оценки P

Рассмотрим, например, задачу определения графа на планарность. Для этого существует два критерия: критерий Понтрягина-Куратовского: граф планарен $\Leftrightarrow$ в нем нет подграфов, гомеоморфных $K_5, K_{3,3}$. Также, существует критерий Вагнера: граф планарен $\Leftrightarrow$ в нем нет миноров $K_5, K_{3,3}$ (минор — граф, полученный из исходного удалением и стягиванием ребер). Рассмотрим свойства, которые сохраняются при удалении и стягивании ребер.

Теорема 1.1 (Робертсона-Сеймура). 1. Для любого свойства, аналогичного планарности выполнен аналог критерия Вагнера с конечным числом запрещенных миноров.

2. Наличие такого минора проверяется за полиномиальное время

Следствие. Любое такое свойство лежит в классе **P**.

Но проблема в том, что мы не знаем миноров, которые необходимо проверить, чтобы найти проверить выполнение данного свойства.

1.3 Другие классы задач

Определение 1.6. $\mathbf{QP} = \mathbf{DTIME}(2^{\text{poly}(\log(n))}) = \bigcup_{c=1}^{\infty} \mathbf{DTIME}(2^{(\log n)^c})$

Определение 1.7. $\mathbf{E} = \mathbf{DTIME}(2^{O(n)}) = \bigcup_{c=1}^{\infty} \mathbf{DTIME}(2^{cn})$

Определение 1.8. $\mathbf{EXP} = \mathbf{DTIME}(2^{\text{poly}(n)}) = \bigcup_{c=1}^{\infty} \mathbf{DTIME}(2^{n^c})$

Определение 1.9. $\mathbf{EE} = \mathbf{DTIME}(2^{2^{cn}}) = \bigcup_{c=1}^{\infty} \mathbf{DTIME}(2^{2^{cn}})$

1.4 Асимптотики различных задач

Пример. $\mathbf{LOG} - \mathbf{CLIQUE} = \{G \mid \omega(G) \geq \log_2 n\} \in \mathbf{QP}$. Является квазиполиномиальной, т.к. $C_n^{\log n} \leq n^{\log n}$, т.е. полный перебор осуществляется за квазиполином

Пример (Задача о доминирующем множестве в турнире). непон

Пример. $\mathbf{GI} = \{(G_1, G_2) : G_1 \cong G_2\} \in \mathbf{QP}$ (изоморфность графов).

Пример. $\mathbf{3COL} = \{G : \chi(G) \leq 3\} \in \mathbf{E}$

Теорема 1.2 (об иерархии по времени). Если $f \ll g \Rightarrow \mathbf{DTIME}(f(n)) \subsetneq \mathbf{DTIME}(g(n))$

2 Класс NP

Сейчас будем рассматривать модель вычислений — *недетерминированную машину Тьюринга* или НМТ. В отличие от обычной машины Тьюринга, функция перехода теперь многозначна (по аналогии с ДКА и НКА).

Соответственно, время работы такой машины Тьюринга — $\text{time}_M(x) = \max \# \text{шагов по всем вариантам перехода}$.

Замечание. Можем считать, что дерево переходов двоичное. Действительно, размер ветвления ограничено мощностью $|\Sigma| \cdot |Q| \cdot |\{N, R, L\}|$ — некоторая константа, не зависящая от входа. Тогда каждое m -ветвление можно заменить $\log_2 m$ 2-ветвлениями.

Ответ данной машины будем понимать следующее:

$$M(x) = \begin{cases} 1, & \text{существует принимающая ветка} \\ 0, & \text{иначе} \end{cases}$$

Замечание. Ответ вычисляется как дизъюнкция по всем результатам работы машины

Определение 2.1. $\mathbf{NTIME}(t(n))$ — класс языков, распознаваемых на НМТ за $O(t(n))$ шагов

Определение 2.2. $\mathbf{NP} = \bigcup_{c=1}^{\infty} \mathbf{NTIME}(n^c)$

Определение 2.3. $\mathbf{NE} = \bigcup_{c=1}^{\infty} \mathbf{NTIME}(2^{cn})$

Определение 2.4. $\mathbf{NEXP} = \bigcup_{c=1}^{\infty} \mathbf{NTIME}(2^{n^c})$

Замечание. $\mathbf{NTIME}(t(n)) \subset \mathbf{DTIME}(2^{t(n)})$

Замечание. $\mathbf{NP} \subset \mathbf{EXP}$ — за время $\mathbf{EXP}$ можно построить все дерево и вычислить ответ по определению.

2.1 Определение через сертификат

Теорема 2.1. $A \in \mathbf{NP} \Leftrightarrow \exists V(x, s) - \text{ДМТ, т.ч. } x \in A \Leftrightarrow \exists s : V(x, s) = 1 \text{ и } V(x, s) \text{ работает за } \text{poly}(|x|)$

Доказательство.

$\Leftarrow$ Рассмотрим следующую НМТ, которая сначала печатает все возможные варианты s (достаточно написать полиномиальное количество символов, т.к. больше V не сможет прочесть), а потом на входе x, s запускает V . Таким образом, Получили машину M , которая в какой-то ветке напечатает нужный сертификат s и выведет $V(x, s) = 1$.

$\Rightarrow$ Возьмем в качестве сертификата код нужной ветки в машине M (0, если надо идти вправо, 1, если влево). Оно и будет нашим сертификатом s . Машина V будет спускаться, в соответствии с сертификатом, по дереву переходов. Тогда сертификат существует $\Leftrightarrow$ существует принимающая ветвь $\Leftrightarrow A \in \mathbf{NP}$.

□

Упражнение. Сформулировать и доказать аналогичную теорему для классов $\mathbf{NE}$, $\mathbf{NEXP}$

Утверждение 2.1. 1. $A, B \in \mathbf{P} \Rightarrow A \cap B, A \cup B, \bar{A} \in \mathbf{P}$

2. $A, B \in \mathbf{NP} \Rightarrow A \cap B, A \cup B \in \mathbf{NP}$

Замечание. Вообще говоря, $\bar{A} \in \mathbf{NP}$ — открытый вопрос. Нельзя просто инвертировать значение машины M (пусть мы получим машину $\bar{M}$): тогда $x \in \bar{A} \Leftrightarrow$ все ветки $\bar{M}$ принимающие, а это не то, что мы хотим. Таким образом, мы приходим к следующему определению:

Определение 2.5. $\mathbf{coNP} = \{A \in \{0, 1\}^* : \bar{A} \in \mathbf{NP}\}$.

Замечание. Аналогично можно доказать, что $A \in \mathbf{coNP}$ тогда и только тогда, когда ответ вычисляется как конъюнкция всех результатов работы машины или тогда и только тогда, когда $\exists V : x \in A \Leftrightarrow \forall s V(x, s) = 1$ и V вычисляется полиномиально от длины x .

Пример. 1. $\mathbf{SAT} = \{\varphi : \exists x : \varphi(x) = 1\} \in \mathbf{NP}$.

Также имеет смысл рассмотреть двойственную задачу (к задаче опровержимости формулы):

2. $\mathbf{TAUT} = \{\varphi : \exists x : \varphi(x) = 1\} \in \mathbf{coNP}$

Замечание. Несмотря на доказанную теорему о полноте, вывод не будет являться сертификатом. Действительно, вывод, вообще говоря, не обязан быть полиномиальным и, в таком случае, машина не сможет полностью его прочесть (т.к. работает полиномиально от $|x|$)

Отдельный интерес у людей науки представляет множество $(\mathbf{coNP} \cap \mathbf{NP}) \setminus \mathbf{P}$. Рассмотрим следующую задачу:

Определение 2.6. $\mathbf{FACTORING} = \{(n, a, b) : \exists d \in (a, b) : d - \text{простое и } n:d\}$

Утверждение 2.2. $\mathbf{FACTORING} \in \mathbf{NP} \cap \mathbf{coNP}$.

Доказательство.

$\in \mathbf{NP}$ d — сертификат

$\in \mathbf{coNP}$ сертификат — разложение на простые, каждое из которых $\notin (a, b)$.

□

2.2 Некоторые следствия из определений

Утверждение 2.3. $\mathbf{P} = \mathbf{NP} \Leftrightarrow \mathbf{P} = \mathbf{coNP}$

Утверждение 2.4. *Следует из того, что $\mathbf{coP} = \mathbf{P}$ ($\mathbf{P}$ замкнут относительно дополнения).*

Замечание. Тем не менее, может быть, что $\mathbf{P} \neq \mathbf{NP}$, но $\mathbf{NP} = \mathbf{coNP}$

Определение 2.7. $A \leq_p B$ сводится по Карпу (сводится полиномиально), если $\exists$ всюду полиномиально вычислимая от $|x|$ функция $f(x)$, такая, что $x \in A \Leftrightarrow f(x) \in B$.

Определение 2.8. $\text{INDSET} = \{(G, k) : \text{в графе } G \text{ есть антиклика из } k \text{ вершин}\}$

Пример. $\text{CLIQUE} \leq_p \text{INDSET}$. Действительно, $f(G, k) = f(\overline{G}, k)$ (дополнение по ребрам).

Определение 2.9. $4\text{COL} = \{G : \exists \text{правильная раскраска в 4 цвета}\}$

Пример. $4\text{COL} \leq_p \text{SAT}$. Мы так уже делали на матлоге, когда сводили некоторые задачи к задачам выполнимости формулы. Для каждой вершины заведем две переменные p_i, q_i , отвечающие за цвет. Нам нужно для каждого ребра записать, что две вершины, являющиеся его концами, имеют разный цвет, и взять конъюнкцию, т.е:

$$\bigwedge_{(i,j) \in E} (p_i \neq p_j) \vee (q_i \neq q_j)$$

Размер данной формулы будет полиномиальным относительно размера графа.

Замечание (Свойства $\leq_p$).

1. $A \leq_p B, B \leq_p C \Rightarrow A \leq_p C$
2. $A \leq_p B, B \in \mathbf{P} \Rightarrow A \in \mathbf{P}$
3. $A \leq_p B, B \in \mathbf{NP} \Rightarrow A \in \mathbf{NP}$
4. $A \leq_p B \Rightarrow \overline{A} \leq_p \overline{B}$

Определение 2.10. Задача $B \in \mathbf{NPH}$ ($\mathbf{NP}$ -трудной), если $\forall A \in \mathbf{NP} : A \leq_p B$.

Определение 2.11. $\mathbf{NPC} = \mathbf{NP} \cap \mathbf{NPH}$ ($\mathbf{NP}$ -полные)

Следствие.

1. $B \in \mathbf{NPH}, B \leq_p C \Rightarrow C \in \mathbf{NPH}$
2. $B \in \mathbf{NPC}, B \leq_p C, C \in \mathbf{NP} \Rightarrow C \in \mathbf{NPC}$

Утверждение 2.5.

1. $P \cap NPH \neq \emptyset \Rightarrow P = NP$
2. $coNP \cap NPH \neq \emptyset \Rightarrow NP = coNP$

Доказательство.

2. $B \in NPH, coNP \Rightarrow \bar{B} \in NP$. Теперь, если $A \leq_p B \Rightarrow \bar{A} \leq_p \bar{B}$. Отсюда получаем, что $\bar{A} \in NP$ и тогда $NP \subset coNP$. Тогда:

$$S \in coNP \Rightarrow \bar{S} \in NP \Rightarrow \bar{S} \in coNP \Rightarrow S \in NP$$

□

Утверждение 2.6. $A \in P, B, \bar{B} \neq \emptyset \Rightarrow A \leq_p B$

Доказательство. Рассмотрим

$$f(x) = \begin{cases} \in B, x \in A \\ \notin B, x \notin A \end{cases}$$

□

Следствие. $P = NP \Rightarrow NPC = P \setminus \{\emptyset, \Sigma^*\}$

Определение 2.12. $TMSAT = \{(M, x, 1^t) : \exists y \text{ } M(x, y) \text{ и работает за } \leq t \text{ шагов}\}$.

Утверждение 2.7. $TMSAT \in NP$

Доказательство. Сертификат — y , верификатор — УМТ

□

Утверждение 2.8. $TMSAT \in NPC$

Доказательство. Принадлежность NP уже доказали, докажем принадлежность NPC . Пусть $A \in NP$. По определению: $x \in A \Leftrightarrow \exists s : V(x, s) = 1$. Положим M — машину Тьюринга, вычисляющую V , $t(n)$ — время работы V для $|x| = n$. Тогда положим $f(x) = (M, x, 1^{t(|x|)})$ и получим, что $A \leq_p TMSAT$. □

Определение 2.13. $3SAT = \{\varphi | \varphi \text{ — выполнимая 3-КНФ}\}$.

Теорема 2.2 (Преобразование Цейтина). $SAT \leq_p 3SAT$.

Доказательство. Для каждой подформулы φ_i заведем свою переменную p_i . Тогда достаточно записать, конъюнкцию формул следующего вида: $q_i \equiv q_j * q_k$, если $\varphi_j = \varphi_j * \varphi_k$. Осталось для каждой такой формулы написать 3-КНФ. □

Напоминание. Наша модель вычислений — одноленточную ДМТ, где лента бесконечная вправо. Пусть Γ — ленточный алфавит, Q — множество состояний, $Q \cap \Gamma = \emptyset$. Пусть $|Q \cup \Gamma| \in (2^{k-1}, 2^k]$, тогда будем использовать k бит для кодирования элементов $Q \cup \Gamma$. Конфигурация ДМТ — строка $AqaB$, $q \in Q$, $A, B \in \Gamma^*$, $a \in \Gamma$.

Определение 2.14. Беспрефиксный код — набор слов, где ни одно из слов не является началом другого

Определение 2.15. Беспрефиксное кодирование — функция, такая, что

Пусть также x — вход, $|x| = n$, машина V работает за $\leq t(n)$ шагов $\Rightarrow V$ использует $\leq t(n)$ ячеек $\Rightarrow$ длина любой конфигурации $\leq t(n) + c$. Также будем считать, что машина имеет два завершающих состояния: q_{accept}, q_{reject} , причём после того, как она пришла в одно из них, она может ещё какое-то время передвигаться по ленте, ничего не меняя.

Теорема 2.3 (Кука-Левина). $SAT \in NPC$

Доказательство. Рассмотрим V — верификатор SAT . Покажем, как написать следующую формулу: "из данных x , данных y машина придет в q_a ". Рассмотрим следующую таблицу (индекс столбца — номер ячейки, индекс строки — индекс в конфигурации):

	0	1	2	...	$t(n)$
0					
1					
2					
$\vdots$					
$t(n)$					

Пусть p_{ij} — k -бит, которые стоят в ij -ой клетке данной таблицы. Запишем формулу, утверждающую, что таблица корректная для V . Тогда наша формула будет иметь вид: $\varphi = \varphi_{start} \wedge \varphi_{accept} \wedge \varphi_{step}$, где:

1. φ_{start} определяет, что изначально машина находилась в корректной конфигурации (изначальная конфигурация $q_1 x \# y \# \dots \#$), т.е., что $p_0 = q_1 x \# y \# \dots \#$:

$$\begin{aligned} \varphi_{start} = & \underbrace{(p_{00} = q_1)}_{q_1} \wedge \underbrace{(p_{01} = x_1) \wedge \dots \wedge (p_{0n} = x_n)}_x \wedge \underbrace{(p_{0,n+1} = \#)}_{\#} \wedge \\ & \wedge \underbrace{(p_{0,n+2} = 0 \vee p_{0,n+2} = 1) \wedge \dots \wedge (p_{0,n+m+1} = 0 \vee p_{0,n+m+1} = 1)}_y \wedge \\ & \wedge \underbrace{(p_{0,n+m+2} = \#) \wedge \dots \wedge (p_{0,t(n)} = \#)}_{\# \dots \#} \end{aligned}$$

2. φ_{accept} проверяет, что в таблице есть хотя бы одно завершающее состояние, т.е., что наша машина завершилась и приняла слово.

$$\varphi_{accept} = \bigvee_{j=0}^{t(n)} (p_{t(n),j} = q_{accept})$$

3. φ_{step} проверяет, что в машина совершала все переходы корректно. Действительно, для таблицы символ p_{ij} однозначно восстанавливается по $p_{i-1,j-1}, p_{i-1,j}, p_{i-1,j+1}, p_{i-1,j+2}$:

	$p_{i-1,j-1}$	$p_{i-1,j}$	$p_{i-1,j+1}$	$p_{i-1,j+2}$
		p_{ij}		

Тогда

$$\varphi_{step} = \bigwedge_{i=1}^{t(n)} \bigwedge_{j=1}^{t(n)} (p_{ij} = f(p_{i-1,j-1}, p_{i-1,j}, p_{i-1,j+1}, p_{i-1,j+2}))$$

Где f — функция, которая восстанавливает по $p_{i-1,j-1}, p_{i-1,j}, p_{i-1,j+1}, p_{i-1,j+2}$ значение p_{ij} .

Тогда формула выполнима $\Leftrightarrow V$ приняло изначальную формулу $\Rightarrow$ получили требуемое $\square$

Определение 2.16. NAE — 3SAT — Дана 3-КНФ. Есть ли набор значений, такой, что в каждой скобке есть истинные и ложные литералы?

Утверждение 2.9. 3SAT $\leq_p$ NAE — 3SAT

Доказательство. Рассмотрим преобразование:

$$(a_i \vee b_i \vee c_i) \mapsto (a_i \vee b_i \vee x_i) \wedge (\neg x_i \vee c_i \vee z)$$

Где x_i уникально для каждого i , а z одно для всей формулы.

Покажем, что исходная формула φ выполнима тогда и только тогда, когда построенная формула ψ принадлежит NAE — 3SAT.

В одну сторону: если φ выполнима, выберем выполняющее назначение и положим $z = 0$. Для каждой скобки $(a_i \vee b_i \vee c_i)$ подберём x_i так, чтобы обе скобки $(a_i \vee b_i \vee x_i)$ и $(\neg x_i \vee c_i \vee 0)$ удовлетворяли условию NAE. Значение x_i можно взять из следующей таблицы истинности:

a_i	$\vee$	b_i	$\vee$	c_i	$(a_i \vee b_i \vee c_i)$	$\wedge$	$(\neg x_i \vee c_i \vee z)$
0		0		1	0		0
0		1		0	0		1
0		1		1	0		1
1		0		0	1		0
1		0		1	1		0
1		1		0	1		0
1		1		1	1		1

Таким образом, полученное назначение даёт решение NAE — 3SAT для ψ .

В обратную сторону: пусть ψ имеет решение в смысле NAE — 3SAT. Если в этом решении $z = 1$, то инвертируем все переменные (включая z) — полученный набор также будет решением, поскольку условие NAE инвариантно относительно отрицания. Поэтому можно считать, что $z = 0$. Тогда для каждой исходной скобки $(a_i \vee b_i \vee c_i)$ имеем две скобки: $(a_i \vee b_i \vee x_i)$ и $(\neg x_i \vee c_i \vee 0)$. Из условия NAE для второй скобки при $z = 0$ получаем, что $\neg x_i \vee c_i$ истинно (иначе все литералы были бы ложны). Для первой скобки условие NAE означает, что не все три литерала равны. Совокупность этих условий влечёт истинность $a_i \vee b_i \vee c_i$: действительно, если бы $a_i = b_i = c_i = 0$, то из первой скобки следовало бы $x_i = 1$ (чтобы не все нули), но тогда вторая скобка стала бы $(\neg 1 \vee 0 \vee 0) = (0 \vee 0 \vee 0)$ — все ложны, противоречие. Значит, все исходные скобки истинны, т.е. φ выполнима.

Таким образом, построенное сведение полиномиально и корректно, откуда NAE — 3SAT $\in$ NP. $\square$

Утверждение 2.10. NAE — 3SAT $\leq_p$ 3COL.

Доказательство. По формуле φ с переменными $x_1, \dots, x_n$ и дизъюнктами $C_1, \dots, C_m$ (каждый — тройка литералов) строим граф G :

1. Три вершины T, F, B , соединённые попарно (треугольник).
2. Для каждой переменной x вершины x и $\neg x$, соединённые ребром, и каждая из них соединена с B .
3. Для каждого дизъюнкта $C = (l_1 \vee l_2 \vee l_3)$ вершины c_1, c_2, c_3 , образующие треугольник, и для каждого i вершина c_i соединена с l_j, l_k , где $\{i, j, k\} = \{1, 2, 3\}$.

Докажем, что $\varphi \in \text{NAE} - 3\text{SAT} \iff \chi(G) \leq 3$.

$\Rightarrow$ Пусть v — оценка, в каждом дизъюнкте есть и истинный, и ложный литерал. Красим T, F, B в разные цвета. Вершину x красим цветом T , если $v(x) = 1$, и F иначе; $\neg x$ — противоположным цветом. В каждом треугольнике дизъюнкта цвета c_i подбираем так: поскольку среди l_1, l_2, l_3 есть оба цвета, можно назначить c_i цвет, отличный от цветов двух смежных литералов (существование такой раскраски проверяется перебором случаев).

$\Leftarrow$ Пусть G правильно раскрашен в 3 цвета. Можно считать, что цвета T, F, B различны. Тогда x и $\neg x$ имеют цвета T и F (так как оба смежны с B и между собой). Положим $v(x) = 1$, если x цвета T , иначе 0. В каждом дизъюнкте C вершины c_1, c_2, c_3 имеют три разных цвета. Каждая c_i смежна с двумя литералами, поэтому эти литералы не могут иметь цвет c_i . Значит, среди трёх литералов не все одинакового цвета.

Таким образом, сведение корректно и выполняется за полиномиальное время. $\square$

Определение 2.17. EXACTSETCOVER — следующая задача: пусть даны $S_1, \dots, S_m \subset \{1, 2, \dots, n\}$, надо понять, существует ли $i_1, \dots, i_k$ и $\bigsqcup_{j=1}^k S_{i_j} = \{1, \dots, m\}$

Утверждение 2.11. $3\text{SAT} \leq_p \text{EXACTSETCOVER}$.

Доказательство. По формуле φ с переменными $x_1, \dots, x_n$ и дизъюнктами $C_1, \dots, C_m$ (каждый — тройка литералов) строим множество U и семейство подмножеств $\mathcal{F}$.

1. Элементы: для каждой переменной x_i : x_i и $\neg x_i$; для каждого дизъюнкта C_j : c_{j1}, c_{j2}, c_{j3} и d_{C_j} .
2. Множества:
 - (а) Для каждого литерала l (т.е. x_i или $\neg x_i$) множество $L_l = \{l\} \cup \{c_{jk} \mid \text{литерал } l \text{ входит в } C_j \text{ на } k\}$
 - (б) Для каждого дизъюнкта C_j и каждого непустого $T \subseteq \{1, 2, 3\}$, $T \neq \{1, 2, 3\}$ (т.е. $|T| \leq 2$), множество $M_{j,T} = \{d_{C_j}\} \cup \{c_{jk} \mid k \in T\}$.

Утверждаем: φ выполнима $\iff$ существует точное покрытие U выбранными множествами.

$\Rightarrow$ Пусть v — выполняющая оценка. Выберем L_{x_i} , если $v(x_i) = 1$, иначе $L_{\neg x_i}$. Эти множества покрывают все элементы $x_i, \neg x_i$ и все c_{jk} , соответствующие истинным литералам. Для каждого C_j пусть S_j — множество позиций истинных литералов. Тогда $S_j \neq \emptyset$. Пусть $T_j = \{1, 2, 3\} \setminus S_j$ — позиции ложных литералов ($|T_j| \leq 2$). Выберем M_{j,T_j} (если $T_j = \emptyset$, берём $M_{j,\emptyset}$ — множество $\{d_{C_j}\}$). Эти множества попарно не пересекаются с выбранными L и между собой и покрывают все оставшиеся c_{jk} и d_{C_j} .

$\Leftarrow$ Пусть есть точное покрытие $\mathcal{C}$. Для каждого i ровно одно из $L_{x_i}, L_{\neg x_i}$ принадлежит $\mathcal{C}$ (иначе x_i или $\neg x_i$ не покрыты). Задаём $v(x_i) = 1$, если $L_{x_i} \in \mathcal{C}$, иначе 0. Для каждого C_j элемент d_{C_j} покрыт некоторым $M_{j,T}$, $T \subseteq \{1,2,3\}$, $|T| \leq 2$. Тогда c_{jk} при $k \in T$ покрыты этим $M_{j,T}$, а c_{jk} при $k \notin T$ должны быть покрыты L_l , т.е. соответствующие литералы истинны. Так как $T \neq \{1,2,3\}$, имеем $k \notin T$ хотя бы для одного k , значит, в C_j есть истинный литерал.

Следовательно, сведение корректно и полиномиально. $\square$

2.3 Задача о рюкзаке и её NP-полнота

Определение 2.18.

$$\text{KNAPSACK} = \left\{ (w_1, \dots, w_n, W, p_1, \dots, p_n, P) : \exists \alpha \in \{0, 1\}^n : \sum_{i=1}^n \alpha_i w_i \leq W, \sum_{i=1}^n \alpha_i p_i \geq P \right\}$$

По сути, по набору предметов w_i, p_i , где первое — вес, второе — стоимость требуется понять, сможем ли мы их поместить в рюкзак вместимости W , чтобы суммарная стоимость была не меньше, чем P .

Определение 2.19. $\text{SUBSET} - \text{SUM} = \{(m_1, \dots, m_n, M) : \exists \alpha \in \{0, 1\}^n : \sum_{i=1}^n \alpha_i m_i = M\}$

Замечание. При $p_i = w_i = m_i, P = W = M$ задача $\text{SUBSET} - \text{SUM}$ эквивалентна задаче KNAPSACK .

Определение 2.20. Частный случай задачи B — это $B \cap S$ для некоторого $S \in P$.

Утверждение 2.12. Частный случай $B \cap S \in \text{NPC} \Rightarrow B \in \text{NPC}$.

Доказательство. Зафиксируем $A \in \text{NP}$. Из определения NP -полноты, существует такая f , что:

$$x \in A \Leftrightarrow f(x) \in B \cap S$$

Рассмотрим (пусть $x_0 \notin B$):

$$f'(x) = \begin{cases} f(x), & f(x) \in S \\ x_0, & f(x) \notin S \end{cases}$$

Эта функция доказывает $A \leq_p B$ $\square$

Утверждение 2.13. $\text{EXACTSETCOVER} \leq_p \text{SUBSET} - \text{SUM}$.

Доказательство. Рассмотрим для каждого S_i строку $a_i = \underbrace{0100 \dots 010}_n$, где $a_{ij} = 1 \Leftrightarrow j \in S_i$

(по сути, маску множества S_i). Рассмотрим данную строку как число в $(n+1)$ -ичной системе счисления. Положим $M = \underbrace{11 \dots 1}_n$ и рассмотрим его как число в $(n+1)$ -ичной

системе счисления. Тогда существует дизъюнктивное покрытие множествами $\Leftrightarrow \exists a_{i_1} + \dots + a_{i_k} = M$ (перехода через разряд не будет, поэтому в каждом разряде должно быть ровно по одному a_{i_j} , который его заполняет). $\square$

Определение 2.21. SETCOVER — то же самое, что и EXACTSETCOVER , только S_i могут пересекаться, но на их количество есть ограничение.

Определение 2.22. $\text{HITTING} - \text{SET} = \{(S_1, \dots, S_n, k) : \exists X = \{x_1, \dots, x_k\} : \forall i : X \cap S_i \neq \emptyset\}$

Утверждение 2.14. $\text{HITTING} - \text{SET} \in \text{NPC}$.

Первое доказательство. Является частным случаем задачи VERTEXCOVER □

Второе доказательство. $3\text{SAT} \leq_p \text{HITTING} - \text{SET}$ □

Доказательство. Переменной p сопоставим множество $\{p_0, p_1, p\}$. Скобке $(l_1 \vee l_2 \vee l_3)$ сопоставим множество $\{l_1, l_2, l_3\}$. Также положим k — количество переменных. Нетрудно проверить, что данное преобразование реализует полиномиальное сведение. □

Определение 2.23. QUADEQ — системы квадратных уравнений. Пусть $A_1, \dots, A_k \in \{0, 1\}^{n \times n}$, $b_1, \dots, b_k \in \{0, 1\}$. Задача состоит в том, существует ли $(x_1, \dots, x_n)^T$ такой, что $\forall i = 1, \dots, k : x^T A_i x = b_i$

Утверждение 2.15. $\text{SAT} \leq_p \text{QUADEQ}$

Доказательство. Доказательство аналогично преобразованию Цейтина. Для каждой подформулы заводим новую переменную и для нее пишем многочлен жегалкина второй степени:

$$\begin{array}{ll} p = q \wedge r & p^2 \oplus qr = 0 \\ p = \neg q & p^2 \oplus q^2 = 1 \\ p = q \vee r & p^2 \oplus q^2 \oplus r^2 \oplus qr = 1 \end{array}$$

И для переменной, отвечающей за исходную формулу пишем уравнение $w^2 = 1$. □

3 Классы задач, связанные с задачами распознавания

3.1 Задача поиска

Рассмотрим задачу поиска: по полиномиальному предикату $V(x, y)$ и данному x требуется найти y такой, что $V(x, y) = 1$ или, в случае, если таких нет, запустить игру Марио.

Однако, в общем случае не всегда задача поиска и распознавания не всегда одинаковы по сложности

Пример. Рассмотрим следующую задачу поиска: "по данному n найти его какой-нибудь простой делитель" и задачу распознавания: "по данному n определить, существуют ли у него простые делители". У второй задачи ответ очевиден ($n = 1$ — нет, иначе — да), первая задача эквивалентна FACTORING , сложность которой не известна.

Теорема 3.1. Любая NPC задачи поиска сводится к задаче распознавания в смысле сводимости по Куку: используем решатель задачи распознавания как к оракулу, запускаем его много раз и проводим некоторые вычисления с ответами.

Пример. Рассмотрим задачу о клике: $x = (G, k)$, $y = S \subset V(G)$ и $V(x, y) = \begin{cases} 1, & |y| = k, y \text{ клика} \\ 0, & \text{иначе} \end{cases}$.

Рассмотрим следующую процедуру:

```

find_clique( $G, k$ ) {
  if ( $G, k \notin \text{CLIQUE}$ ) {
    return  $\perp$ 
  }

   $v$  = random vertex from  $G$ 

  if ( $G \setminus \{v\}, k \in \text{CLIQUE}$ ) {
    return find_clique( $G \setminus \{v\}, k$ )
  } else {
    return find_clique( $G \setminus \{v\}, k - 1$ )  $\cup \{v\}$ 
  }
}

```

3.2 Задача подсчета

Также имеет смысл рассмотреть задачи подсчета: по полиномиальному предикату $V(x, y)$ и данному x требуется найти количество y таких, что $V(x, y) = 1$.

Пример. Пусть A — матрица размера $n \times n$ из нулей и единиц.

$$\text{Perm } A = \sum_{\sigma \in S_n} \prod_{i=1}^n A_{i\sigma(i)}$$

По сути, это задача о нахождении количества совершенных паросочетаний для графа, для которого A задает матрицу Татта (матрица смежности между двумя долями). Вопрос о нахождении паросочетания можно находить полиномиально, но вот данная задача уже сложная.

Теорема 3.2. Подсчет $\text{Perm } A$ — **NPС**-задача

3.3 Задача аппроксимации

Пусть дано x . Требуется найти максимальное (минимальное) значение $f(x, y)$ по y . Задача оптимизации для данной задачи звучит так: найти y такой, что $f(x, y) \leq \rho \cdot \max_y f(x, y)$ и $f(x, y) \geq \rho \cdot \min_y f(x, y)$.

И может так случиться, что задача из **NPС**, но ее приближение полиномиально.

Пример. **NPС**-задача: найти вершинную покраску. Приближение: ищем с фактором 2. Достаточно выбрать максимальное паросочетание, берем все вершины и ребра из него.

Однако, бывает так, что даже приближение задачи **NP**-трудно.

4 Полиномиальная иерархия

Определение 4.1. $\text{DP} = \{A \cap B : A \in \text{NP}, B \in \text{coNP}\} = \{A \setminus B : A \in \text{NP}, B \in \text{NP}\}$

4.1 Примеры

Вспомним арифметическую иерархию. Мы строили ПНФ для формул, которые являются вычислимыми. Построим полиномиальную иерархию: оставшаяся часть после кванторов должна быть полиномиально вычислима. Рассмотрим следующие задачи:

1. **Задача о минимизации формулы.** $\text{MINEQFORM} = \{(\varphi, k) : \exists \psi \sim \varphi, |\psi| \leq k\}$. В кванторах эту задачу можно записать так: $\exists \psi \forall x (|\psi| \leq k \wedge \psi(x) = \varphi(x))$. Заметим, что эта формула в ПНФ, причем начинается с $\exists$.
2. **Задачи точной оптимизации.** Например: $\text{CHROMNUM} = \{(G, k) : \chi(G) = k\}$. В кванторах эту задачу можно записать так: $\exists$ правильная раскраска в k цветов $\wedge \forall$ раскраска в $k - 1$ цвет неправильная. Её можно привести к ПНФ, причем начинаться она будет с двух кванторов. Порядок кванторов в данном случае не имеет значения
3. **Обобщенная Рамсеевость.** Число Рамсея определяется следующим образом: $R(s, t) = \min\{N : \text{в любом графе на } N \text{ вершинах есть } s\text{-клика или } t\text{-независимое множество}\}$. Обобщенная Рамсеевость, же задает следующий вопрос: пусть даны 3 графа G, H_1, H_2 , верно ли что при любой раскраске G в 2 цвета найдется подграф первого цвета, изоморфный H_1 или подграф второго цвета, изоморфный H_2 ? Данная задача приводится к ПНФ следующим образом:

$\forall$ раскраски $\exists$ подграф, изоморфизм ...

Здесь уже формула начинается с $\forall$.

4. **Кликовая раскраска.** Существует ли кликовая раскраска графа в k цветов, т.е. такая, что в любой максимальной клике есть вершины разного цвета. Таким образом, в ПНФ будет 2 квантора:

$\exists$ раскраска $\forall$ максимальной клики $\underbrace{\exists 2 \text{ вершины}}_{\text{полиномиально вычислимо}} \dots$

5. **Наследуемая кликовая раскраска.** Аналогично предыдущему, но утверждение должно выполняться для любого подграфа. ПНФ:

$\forall$ подграфа $\exists$ раскраска $\forall$ максимальной клики ...

Определение 4.2. Пусть $U = \{1, \dots, n\}$ и дана система множеств $\{S_1, \dots, S_m\}, S_i \subset U$. Тогда: $\dim_{VC}(S_1, \dots, S_m) = \max\{|T| : \forall X \subset T : \exists i : S_i \cap T = X\}$. Таким образом: $\text{VCDIM} = \{(S_1, \dots, S_m, d) : \dim_{VC}(S_1, \dots, S_m) \geq d\}$.

5. **Размерность Вапника-Червоненкиса.** Задача VCDIM , но с неявным заданием формулы (чтобы можно было записать экспоненциальное количество множеств) через формулу $\varphi(i, x) = \begin{cases} 1, & x \in S_i \\ 0, & x \notin S_i \end{cases}$.

4.2 Формально

Определение 4.3. Σ_k^P — класс языков A , таких, что $\exists V$ — полиномиально вычислимая от длины x , такая, что:

$$x \in A \Leftrightarrow \exists y_1 \forall y_2 \dots \forall y_k V(x, y_1, \dots, y_k)$$

Определение 4.4. Π_k^P — класс языков A , таких, что $\exists V$ — полиномиально вычислимая от длины x , такая, что:

$$x \in A \Leftrightarrow \forall y_1 \exists y_2 \dots \forall y_k V(x, y_1, \dots, y_k)$$

Замечание. 1. $P = \Sigma_0^P = \Pi_0^P$

$$2. \Sigma_i^P, \Pi_i^P \subset \Sigma_{i+1}^P, \Pi_{i+1}^P$$

$$3. \Sigma_1^P = \mathbf{NP}, \Pi_1^P = \mathbf{coNP}$$

$$4. \Sigma_1^P, \Pi_1^P \subset \mathbf{DP} \subset \Sigma_2^P, \Pi_2^P$$

Определение 4.5.

$$\mathbf{PH} = \bigcup_{k=0}^{\infty} \Sigma_k^P = \bigcup_{k=0}^{\infty} \Pi_k^P$$

4.3 Вопрос о коллапсе иерархии

Возникает вопрос: а почему $\mathbf{PH} \neq \mathbf{P}$?

Теорема 4.1. $\mathbf{P} = \mathbf{NP} \Rightarrow \mathbf{P} = \mathbf{PH}$

Идея доказательства. $\mathbf{P} = \mathbf{NP} \Rightarrow \mathbf{P} = \mathbf{coNP}$. Но тогда мы можем убирать последние кванторы в формуле и, таким образом, $\Sigma_i^P, \Pi_i^P = \mathbf{P}$. $\square$

Замечание. $A, B \in \Sigma_k^P \Rightarrow A \cup B, A \cap B \in \Sigma_k^P, A \in \Sigma_k^P \Leftrightarrow \bar{A} \in \Pi_k^P$

Доказательство. $A \in \Sigma_k^P \Leftrightarrow \bar{A} \in \Pi_k^P$ — очевидно из отрицания к формуле A . Докажем, что $A \cup B \in \Sigma_k^P$. Это следует из следующего (если φ, ψ не зависят от y, x соответственно):

$$\forall x \varphi(x) \vee \forall y \psi(y) \Leftrightarrow \forall (x, y) \varphi \vee \psi$$

Утверждение $A \cap B \in \Sigma_k^P$ получается и равенства $A \cap B = \overline{\bar{A} \cup \bar{B}}$. $\square$

Теорема 4.2 (Об условиях коллапса). *Следующие условия эквивалентны:*

$$1. \Sigma_k^P = \Sigma_{k+1}^P$$

$$1'. \Pi_k^P = \Pi_{k+1}^P$$

$$2. \Sigma_k^P = \Pi_k^P, k > 0$$

$$3. \mathbf{PH} = \Sigma_k^P$$

Доказательство.

$$(3) \Rightarrow (1) \quad \Sigma_k^P \subset \Sigma_{k+1}^P \subset \mathbf{PH} = \Sigma_k^P$$

$$(1) \Rightarrow (2) \quad \Pi_k^P \subset \Sigma_{k+1}^P = \Sigma_k^P \Rightarrow \Pi_k^P = \Sigma_k^P. \text{ Теперь:}$$

$$A \in \Sigma_k^P \Rightarrow \bar{A} \in \Pi_k^P \Rightarrow \bar{A} \in \Sigma_k^P \Rightarrow A \in \Pi_k^P \Rightarrow \Sigma_k^P = \Pi_k^P$$

$$(2) \Rightarrow (3) \text{ Рассмотрим } A \in \Sigma_{k+1}^P$$

$$x \in A \Leftrightarrow \exists y_1 \forall y_2 \dots \forall y_{k+1} V(x, y_1, \dots, y_{k+1})$$

Рассмотрим теперь язык

$$Z = \{(x, y_1) : \forall y_2 \dots \forall y_{k+1} V(x, y_1, \dots, y_{k+1})\} \in \Pi_k^P \Rightarrow \Sigma_k^P$$

Но тогда:

$$Z = \{(x, y_1) : \exists z_1 \dots \forall z_k W(x, z_1, \dots, z_k)\}$$

Из чего заключаем:

$$x \in A \Leftrightarrow \underbrace{\exists y_1 \exists z_1 \dots \forall z_k W(x, z_1, \dots, z_k)}_{\exists(y_1, z_1)}$$

Получили, что $A \in \Sigma_k^P$. Тогда $\Sigma_{k+1}^P = \Pi_{k+1}^P = \Sigma_k^P$. Отсюда по индукции получаем, что $\Sigma_{k+l}^P = \Pi_{k+l}^P = \Sigma_k^P \Rightarrow \mathbf{PH} = \Sigma_k^P$

□

Замечание. $A \leq_P B, B \in \Sigma_k^P \Rightarrow A \in \Sigma_k^P$.

4.4 Полнота в полиномиальной иерархии

Можно по аналогии определить Σ_k^P -полные, Π_k^P -полные и $\mathbf{PH}$ -полные задачи:

Определение 4.6. $A \in \Sigma_k^P$ называется полной, если $\forall B \in \Sigma_k^P : B \leq_P A$

Определение 4.7. $A \in \Pi_k^P$ называется полной, если $\forall B \in \Pi_k^P : B \leq_P A$

Определение 4.8. $A \in \mathbf{PH}$ называется полной, если $\forall B \in \mathbf{PH} : B \leq_P A$

Замечание. Из теоремы об условиях коллапса, получаем, что $\mathbf{PH}$ -полные задачи существуют в том и только том случае, когда иерархия коллапсирует.

Обобщим следующие задачи для классов $\mathbf{P}, \mathbf{NP}$:

$$\text{SAT} = \{\varphi \mid \exists x : \varphi(x) = 1\}$$

$$\text{TAUT} = \{\varphi \mid \forall x : \varphi(x) = 1\}$$

Таким образом, просто добавим кванторы:

$$\Sigma_k \text{SAT} = \{\varphi \mid \exists x_1 \forall x_2 \dots \forall x_k : \varphi(x)\}$$

$$\Pi_k \text{SAT} = \{\varphi \mid \exists x_1 \forall x_2 \dots \forall x_k : \varphi(x)\}$$

Можно смотреть на данную задачу, как на "Игру в формулу": Алиса и Боб играют по следующим правилам: Алиса выбирает x_1 , Боб выбирает x_2 и так далее. Если в итоге получилось, что $\varphi = 1$, то победила Алиса, иначе — Боб (для задачи $\Pi_k\text{SAT}$ — наоборот). Таким образом, задача $\Sigma_k\text{SAT}$, по сути, утверждает, что у Алисы есть выигрышная стратегия, а $\Pi_k\text{SAT}$ — что у Боба.

Лемма 4.1. $\Sigma_k\text{SAT}$ полна в $\Sigma_k^P \Leftrightarrow \Pi_k\text{SAT}$ полна в Π_k^P

Доказательство. Достаточно доказать только в одну сторону (в другую аналогично). Докажем вправо. $\varphi \notin \Sigma_k\text{SAT} \Leftrightarrow \neg\varphi \in \Pi_k\text{SAT}$. Тогда, если $A \in \Pi_k^P \Rightarrow \bar{A} \in \Sigma_k^P$. Из этого следует, что $\bar{A} \leq_P \Sigma_k\text{SAT}$. Но тогда:

$$x \in \bar{A} \Leftrightarrow f(x) \in \Sigma_k\text{SAT}$$

$$x \in A \Leftrightarrow f(x) \notin \Sigma_k\text{SAT}$$

$$x \in A \Leftrightarrow f(x) \in \Pi_k\text{SAT}$$

$$A \leq_P \Pi_k\text{SAT}$$

□

Лемма 4.2. $\Sigma_k\text{SAT}$ полно в Σ_k^P при нечетных k . $\Pi_k\text{SAT}$ полно в Σ_k^P при четных k .

Доказательство. Пусть k нечетно, $A \in \Sigma_k^P$. $x \in A \Leftrightarrow \exists y_1 \forall y_2 \dots \exists y_k V(x, y_1, \dots, y_k)$. В таком случае сводимость будет точно такая же как и в теореме Кука-Левина, только последние два получившиеся квантора $\exists x_k \exists$ (заполнение таблицы) мы склеим в один. □

Замечание. Для четных k уже не получится так же доказать, что $\Sigma_k\text{SAT}$ полно в Σ_k^P (т.к. последние кванторы не склеятся в один) и придется использовать первую лемму

Таким образом, мы получаем следствие:

Следствие. $\Sigma_k\text{SAT}, \Pi_k\text{SAT}$ полны в Σ_k^P, Π_k^P соответственно.

Следствие. $\forall k \exists \Sigma_k^P$ -полные и Π_k^P -полные задачи.

4.5 Альтернирующие Машины Тьюринга

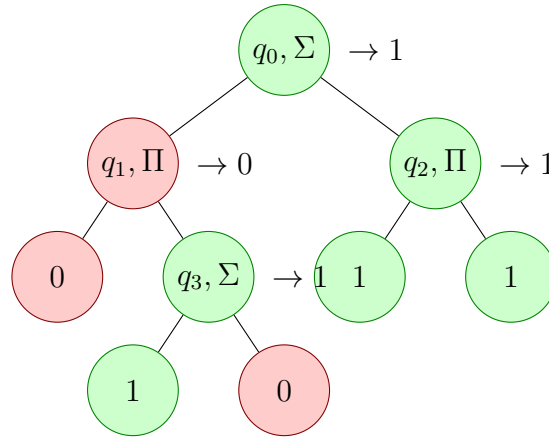
Определение 4.9. Альтернирующая машина Тьюринга — это такая, у которой

1. Функция переходов многозначная
2. Состояния подразделяются на Σ -состояния и Π -состояния

Вычисление ответа машины происходит следующим образом: заметим, что по завершении работы, получается некоторое дерево вычисления нашей машины. Результат вычислений определяем следующим образом:

1. Для листа q_{accept} означат результат 1, а q_{reject} — результат 0
2. Для произвольной вершины в случае Σ -состояния результатом будет дизъюнкция ответов всех её сыновей, иначе — конъюнкция.

Пример (Вычисление ответа на Альтернирующей Машине Тьюринга).



Замечание. НМТ — частный случай АМТ (достаточно сделать все состояния Σ -состояниями).

Время работы АМТ, соответственно, определяется как длина самой большой ветви. Число альтернатив — максимальное количество переключений $\Sigma \leftrightarrow \Pi$.

Определим следующие классы языков:

Определение 4.10. $\Sigma_k \text{TIME}(t(n))$ — класс языков, распознаваемых на АМТ с начальным состоянием Σ , временем работы $O(t(n))$ и числом альтернатив $\leq k$

Определение 4.11. $\Pi_k \text{TIME}(t(n))$ — класс языков, распознаваемых на АМТ с начальным состоянием Π , временем работы $O(t(n))$ и числом альтернатив $\leq k$

Утверждение 4.1.

$$\Sigma_k^P = \bigcup_{c=1}^{\infty} \Sigma_k \text{TIME}(n^c)$$

$$\Pi_k^P = \bigcup_{c=1}^{\infty} \Pi_k \text{TIME}(n^c)$$

Утверждение 4.2. Докажем для Σ_k^P

- ⊆ Рассмотрим следующую АМТ: в Σ -состоянии пишем y_1 , переходим в Π -состояние, пишем в нем y_2 и так далее.
- ⊇ Положим y_1 — номер ветви вдоль Σ -состояния, y_2 — номер ветви вдоль Π -состояния и так далее. Машина V будет просто вычислять результат вдоль ветви $y_1, \dots, y_k$.

А что если время работы полиномиально, но число альтернатив не ограничено? Такой класс называется **AP** и он совпадает с классом **PSPACE**, т.е. с классом языков, распознаваемых на полиномиальной памяти.

5 Пространственная сложность

В данном разделе мы перейдем к новой модели вычисления. Мы будем рассматривать машину Тьюринга с двумя лентами: на одной будет записан неизменяемый вход (машина не может менять эту ленту, только перемещаться по ней). Вторая лента будет обычной лентой машины Тьюринга.

Замечание. Конфигурация МТ с двумя лентами содержит информацию о состоянии двух лент и положении двух указателей на эти ленты.

Определение 5.1. Определим используемую память как количество ячеек на рабочей ленте, которые машина посетила ≥ 1 раз

Замечание. Для недетерминированных машин Тьюринга используемая память определяется как максимальная из используемых памяти по всем ветвям работы машины.

Определение 5.2. $\text{DSPACE}(s(n))$ — языки, распознаваемые ДМТ на памяти $O(s(n))$, где n — длина входа.

Определение 5.3. $\text{NSPACE}(s(n))$ — языки, распознаваемые НМТ на памяти $O(s(n))$, где n — длина входа.

Введем еще обозначения:

Определение 5.4. 1. $\mathbf{L} = \text{DSPACE}(\log n)$

2. $\mathbf{NL} = \text{NSPACE}(\log n)$

3. $\mathbf{polyL} = \bigcup_{k=1}^{\infty} \text{NSPACE}((\log n)^k)$

4. $\mathbf{PSPACE} = \bigcup_{k=1}^{\infty} \text{DSPACE}(n^k)$

5. $\mathbf{NPSPACE} = \bigcup_{k=1}^{\infty} \text{NSPACE}(n^k)$

Замечание. Нам не важно, по какому основанию берется логарифм, т.к. $\log_a x = c \cdot \log_b x$

Утверждение 5.1. $\mathbf{PSPACE} \subset \mathbf{EXP}$

Доказательство. В процессе работы не может встретиться двух одинаковых конфигураций, иначе машина бы вошла в бесконечный цикл. Таким образом, общее их количество не превосходит (пусть Γ — ленточный алфавит, Q — множество состояний) $|\Gamma|^{\text{poly}(n)} \cdot |Q| \cdot n \cdot \text{poly}(n)$. Таким образом, получаем, что максимальное время работы — $O(c^{\text{poly}(n)})$ для достаточно большой константы c . $\square$

Утверждение 5.2. $\mathbf{L} \subset \mathbf{P}$

Доказательство. Аналогично считаем количество конфигураций: $|\Gamma|^{\log n} \cdot |Q| \cdot n \cdot \log n = O(\text{poly}(n))$. $\square$

Утверждение 5.3. $\mathbf{NL} \subset \mathbf{P}$

Доказательство. Количество конфигураций тоже является $\text{poly}(n)$. Рассмотрим граф конфигураций работы данной машины: проведем ребро из a в b , если из конфигурации a можно попасть в конфигурацию b за один переход машины (в какой то из ветвей вычисления). Запустим **dfs** из стартовой вершины и получим полиномиальный алгоритм, который полиномиально определяет принадлежность $\mathbf{NL}$ -языку:

1. Строим граф, соответствующий входу x — $O(\text{poly}(n))$
2. Запускаем **dfs**, проверяющий, что мы можем прийти в конечную конфигурацию с ответом 1 — $O(\text{poly}(n))$

□

Теорема 5.1 (Сэвича). Если $s(n) \geq \log n$, то $\mathbf{NSPACE}(s(n)) \subset \mathbf{DSpace}(s(n)^2)$

Доказательство. Число конфигураций считаем аналогично: $|\Gamma|^{s(n)} \cdot |Q| \cdot n \cdot s(n) = N$. Это создает проблему: хранить граф конфигурации слишком дорого. Поэтому введем предикат $\text{Reach}(u, v, k)$ = есть путь $u \rightarrow v$ длины $\leq k$. Таким образом, $\text{Reach}(u, v, N) \Leftrightarrow$ между u, v существует путь (т.к. путь не превышает количества вершин в графе). Будем вычислять $\text{Reach}(u, v, k)$ рекурсивно:

```

Reach(u, v, k) {
  if k = 1 {
    return u = v or u → v за один шаг
  }

  return  $\bigvee_x (\text{Reach}(u, x, \lfloor \frac{N}{2} \rfloor) \wedge \text{Reach}(x, v, \lceil \frac{N}{2} \rceil))$ 
}

```

Каждая конфигурация недетерминированной машины Тьюринга, работающей в пространстве $s(n)$, может быть записана с помощью $O(s(n))$ ячеек (всего их $c^{s(n)}$, поэтому чтобы их занумеровать, нужно $s(n)$ памяти). Число k в вызове **Reach** не превышает общего числа конфигураций $N = |\Gamma|^{s(n)} \cdot |Q| \cdot n \cdot s(n)$, поэтому его двоичная запись требует $O(\log N) = O(s(n))$ бит.

Глубина рекурсии определяется тем, что на каждом шаге параметр k уменьшается примерно вдвое, поэтому число уровней рекурсии равно $O(\log N) = O(s(n))$. На каждом уровне на ленте хранятся: конфигурации u, v , текущая перебираемая конфигурация x и значение k . Все они имеют размер $O(s(n))$. Таким образом, суммарная память, занятая машиной, составляет $O(s(n)) \cdot O(s(n)) = O(s(n)^2)$. □

Следствие. $\mathbf{PSPACE} = \mathbf{NPSPACE}$, $\mathbf{NL} \subset \mathbf{polyL}$

5.1 Задачи из PSPACE

Замечание. В **PSPACE** лежат:

1. РН,

5.2 Задача TQBF

Определение 5.5. $\text{TQBF} = \{\varphi \mid \exists x_1 \forall x_2 \dots \exists x_n \varphi(x_1, \dots, x_n)\}$.

Замечание. $\text{TQBF} \in \mathbf{PSPACE}$

Определение 5.6. **PSPACE**-полные задачи определим аналогично **NP**-полным (сводимость тоже $\leq_p$).

Теорема 5.2. TQBF — **PSPACE**-полный язык.

Доказательство. Принадлежность $\text{TQBF} \in \mathbf{PSPACE}$ нам уже известна. Покажем, что $\forall A \in \mathbf{PSPACE} : A \leq_p \text{TQBF}$

Докажем, что любой язык $A \in \mathbf{PSPACE}$ сводится к **TQBF**. Пусть M — детерминированная машина Тьюринга, распознающая A с использованием памяти $p(n)$, где p —

полином. Для входного слова x длины n число различных конфигураций M не превосходит $N = c^{p(n)}$ для некоторой константы c (зависящей от алфавита и числа состояний). Поэтому конфигурации можно закодировать двоичными строками длины $O(p(n))$.

Построим булеву формулу с кванторами, которая истинна тогда и только тогда, когда M принимает x . Для этого определим семейство формул $\Phi_i(a, b)$, выражающих существование пути из конфигурации a в конфигурацию b длины не более 2^i .

- ▷ Базовый случай $i = 0$: $\Phi_0(a, b)$ истинна, если $a = b$ или $a \rightarrow b$ за один шаг. Это условие можно записать в виде пропозициональной формулы фиксированного размера, проверяющей корректность перехода (аналогично φ_{step} в теореме Кука–Левина).
- ▷ Шаг рекурсии:

$$\Phi_i(a, b) = \exists m \forall u \forall v ((u = a) \wedge (v = m)) \vee ((u = m) \wedge (v = b)) \rightarrow \Phi_{i-1}(u, v)$$

Здесь m, u, v — переменные, кодирующие конфигурации. Смысл: существует промежуточная конфигурация m такая, что для любых u, v , если пара (u, v) совпадает с (a, m) или (m, b) , то выполняется $\Phi_{i-1}(u, v)$. Это эквивалентно тому, что путь разбивается на две половинки.

Рекуррентное построение даёт формулу глубины кванторов $O(i)$. Так как максимальная длина пути не превосходит N , достаточно взять $i = \lceil \log_2 N \rceil = O(p(n))$. Итоговая формула $\Phi_i(C_{\text{start}}, C_{\text{accept}})$ (где C_{start} — начальная конфигурация на входе x , а C_{accept} — единственная принимающая конфигурация) истинна тогда и только тогда, когда M принимает x .

Оценим размер формулы. На каждом уровне мы добавляем константное число кванторов и копируем подформулу Φ_{i-1} дважды. Поэтому размер растёт как $O(p(n)^2)$. Построение выполняется за полиномиальное время (алгоритм рекурсивно выписывает формулу, подставляя соответствующие коды конфигураций). $\square$

5.3 Игра GG и её PSPACE-полнота

Определение 5.7. $\text{GG} = \{(G, s) \mid G \text{ — ориентированный граф, } s \text{ — стартовая вершина, первый игрок}\}$

Теорема 5.3. $\text{TQBF} \leq_p \text{GG}$, следовательно GG PSPACE-полна.

Доказательство. Построим полиномиальное сведение. Пусть $\Phi = Q_1 x_1 Q_2 x_2 \dots Q_k x_k \varphi(x_1, \dots, x_k)$, где φ — 3-КНФ с дизъюнктами $C_1, \dots, C_m$. Построим ориентированный граф G и стартовую вершину s , моделирующие игру, соответствующую кванторной структуре Φ .

Гаджеты.

1. **Гаджет выбора значения переменной.** Для каждой переменной x_i :

- ▷ Если $Q_i = \exists$ (ход первого игрока), создаём вершину a_i (вход гаджета) и две вершины-значения v_i^0, v_i^1 . Проводим рёбра $a_i \rightarrow v_i^0, a_i \rightarrow v_i^1$, а также $v_i^0 \rightarrow a_{i+1}, v_i^1 \rightarrow a_{i+1}$.
- ▷ Если $Q_i = \forall$ (ход второго игрока), добавляем промежуточную вершину b_i . Проводим $a_i \rightarrow b_i, b_i \rightarrow v_i^0, b_i \rightarrow v_i^1$, и далее $v_i^0 \rightarrow a_{i+1}, v_i^1 \rightarrow a_{i+1}$.

Положим $a_1 = s$.

2. **Гаджет проверки выполнимости.** После последнего блока положим $a_{k+1} = check$. Создадим вершины $d_1, \dots, d_m$ (по одной на дизъюнкт) и вершину win . Проведём рёбра $check \rightarrow d_j$ для всех j . Для каждого дизъюнкта $C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$ создадим три вершины-литерала L_{j1}, L_{j2}, L_{j3} (разные для каждого вхождения). Проведём рёбра $d_j \rightarrow L_{jt}$ для $t = 1, 2, 3$. Из каждой L_{jt} проведём ребро в win . Из win нет исходящих рёбер.
3. **Гаджет связи выбора с литералами.** Для каждой вершины-значения v_i^0 проведём рёбра во все вершины L , соответствующие литералу $\neg x_i$ (во всех дизъюнктах). Аналогично, из v_i^1 проведём рёбра во все вершины L , соответствующие литералу x_i .

Корректность.

1. *Моделирование кванторов.* Игроки последовательно проходят гаджеты выбора. В гаджете $\exists$ первый игрок сам выбирает v_i^0 или v_i^1 , фиксируя значение x_i . В гаджете $\forall$ первый игрок вынужден пойти в b_i , после чего второй игрок выбирает значение. Таким образом, после k ходов оказывается посещён ровно один набор вершин v_i^σ , кодирующий оценку переменных, и фишка приходит в $check$.
2. *Проверка выполнимости.* Из $check$ первый игрок выбирает дизъюнкт d_j . Второй игрок выбирает литерал, переходя в L_{jt} . Рассмотрим два случая:
 - ▷ Если выбранный литерал истинен при данной оценке, то соответствующая вершина-значение v_i^σ была посещена на этапе выбора. По построению из v_i^σ есть ребро в L_{jt} . Однако v_i^σ уже посещена, и повторно в неё ходить нельзя. Чтобы обойти это, в стандартной конструкции вместо прямого ребра $L_{jt} \rightarrow v_i^\sigma$ делают следующее: из L_{jt} ведут ребро в новую вершину L' , а из L' — в v_i^σ и в win . Поскольку v_i^σ уже посещена, ход в неё невозможен, зато первый игрок может пойти из L' в win и выиграть.
 - ▷ Если же литерал ложен, то соответствующая вершина-значение не посещалась, и из L_{jt} нет пути в win (или путь ведёт только через непосещённую v_i^σ , которая недостижима), поэтому второй игрок, сделавший ход в L_{jt} , не может сделать следующий ход и проигрывает.

Опуская эти технические детали (они не влияют на полиномиальность), можно считать, что первый игрок выигрывает тогда и только тогда, когда выбранный литерал истинен. Следовательно, второй игрок будет выбирать только ложные литералы, если они есть. Первый игрок выигрывает тогда и только тогда, когда в каждом дизъюнкте найдётся истинный литерал, то есть φ истинна при полученной оценке. Поскольку оценка формируется в соответствии с кванторами, наличие выигрышной стратегии у первого игрока равносильно истинности Φ .

Построение графа требует $O(k + m + \text{число вхождений литералов})$ вершин и рёбер, то есть полиномиально от размера Φ . Таким образом, $\text{TQBF} \leq_p \text{GG}$, а так как TQBF **PSPACE**-полна, то **GG** также **PSPACE**-полна. $\square$

Следствие. $\text{GG} \in \text{PSPACE}$ (тривиально, так как можно рекурсивно перебирать стратегии), и в силу полноты **GG** является **PSPACE**-полной задачей.

6 Классы L, NL

6.1 $L \stackrel{?}{=} NL$

Определение 6.1. $L = DTIME(\log n)$

Определение 6.2. $NL = NTIME(\log n)$

Замечание. $NL \subset L \subset P$

Замечание. Регулярные языки лежат в L

Замечание. Арифметические действия тоже можно вычислять за логарифмическую память. Действительно: постепенно ходим вперед-назад по входу и проверяем, что числа посимвольно совпадают, записываем результат и индекс проверяемой буквы.

Сложение разделяется на три задачи:

1. Проверка корректности вычисления
2. Вычисление i -ого бита
3. Непосредственное вычисление не логарифмической памяти

Понятно, что если мы можем вычислить i -ый бит, то все остальные задачи тоже сможем решить на логарифмической памяти.

Умножение можно сделать следующим образом: пусть p_i — результат i -ой цифры, а c_i — перенос для i -ой цифры. Тогда $p_0 = a_0b_0, p_1 = a_1b_0 + a_0b_1 \bmod 2, c_2 = \left\lfloor \frac{a_1b_0 + a_0b_1}{2} \right\rfloor$ и так далее. Можно заметить, что $c_i \leq n$, поэтому на хранение c_i потребуется на более $O(\log n)$ битов.

Замечание. Язык всех ПСП лежит в L. Действительно, для скобочных последовательностей из одной скобки достаточно завести счетчик, показывающий скобочный баланс.

Для нескольких скобок будем находить для каждой скобки парную ей (баланс для соответствующих скобок должен быть равен 0). Но тогда и для остальных скобок баланс должен быть равен 0, это и проверяем. Для этого достаточно завести несколько счетчиков.

Определение 6.3. $UPATH = \{(G, s, t) : \text{в графе } G \text{ есть путь } s \rightarrow t\}$

Теорема 6.1 (Рейнгольда, 2005). $UPATH \in L$

Доказательство. Очень сложно □

Следствие. Язык всех связных графов также лежит в L (достаточно проверить, что из фиксированной вершины достижимы все остальные). Более того, многие другие свойства неориентированных графов проверяются за логарифмическую память:

- ▷ **Цикличность:** граф содержит цикл тогда и только тогда, когда он не является лесом. Для связного графа это эквивалентно тому, что число рёбер $\geq$ число вершин. Подсчёт рёбер можно выполнить за $O(\log n)$ памяти, перебирая все пары вершин (i, j) с $i < j$ и увеличивая счётчик при наличии ребра.

- ▷ **Двудольность:** граф двудольный $\iff$ в нём нет циклов нечётной длины. Проверка сводится к UPATH в расширенном графе: строим G' с вершинами $(v,0), (v,1)$ и рёбрами $(u,i) - (v,1-i)$ для каждого ребра $(u,v) \in G$. Тогда G двудольный $\iff$ для любого ребра (u,v) нет пути из $(u,0)$ в $(v,0)$ в G' . Поскольку $\text{UPATH} \in \mathbf{L}$, то и двудольность в $\mathbf{L}$.
- ▷ **Дерево:** связный ациклический граф. Связность проверяется через UPATH, а ациклическость — через подсчёт рёбер: дерево имеет ровно $n - 1$ ребро. Унициклические графы (связные, с n рёбрами) не являются деревьями; их отличие фиксируется сравнением с $n - 1$.

После теоремы Рейнгольда и замечаний о задачах в $\mathbf{L}$ добавим:

Определение 6.4. $\text{PATH} = \{(G, s, t) : G \text{ — ориентированный граф, существует путь из } s \text{ в } t\}$

Замечание. В отличие от неориентированного случая, для ориентированных графов PATH не удаётся решить детерминированно за логарифмическую память (предполагается, что $\text{PATH} \notin \mathbf{L}$). Простой вероятностный алгоритм — случайное блуждание по графу — может заикнуться или не найти путь даже при его наличии, так как нет гарантии быстрого достижения цели из-за отсутствия симметрии рёбер. Доказательство Рейнгольда для неориентированных графов существенно опирается на обратимость переходов.

Утверждение 6.1. $\text{PATH} \in \mathbf{NL}$.

Доказательство. Недетерминированная машина Тьюринга угадывает последовательность вершин (или направлений) длины не более n (где $n = |V|$), начиная с s . На каждом шаге проверяет существование ребра из текущей вершины в следующую. Если после не более n шагов достигнута t , машина принимает. Для хранения текущей вершины и счётчика достаточно $O(\log n)$ памяти. $\square$

Определение 6.5. $A \leq_l B \iff \exists f$ — вычислимая на логарифмической памяти, такая, что $x \in A \iff f(x) \in B$.

Логарифмическую вычислимость функции f стоит понимать следующим образом: у f есть три ленты: вход, выход и рабочая. Вход будет readonly, выход будет являться потоком (т.е. старые символы стирать нельзя).

Замечание. Альтернативное определение логарифмической вычислимости: f вычислима за $O(\log n)$ памяти, если $\{(x, l) : |f(x)| \geq l\} \in \mathbf{L}$ и $\{(x, i) : (f(x))_i = 1\} \in \mathbf{L}$. Это определение корректно, т.е. эквивалентно стандартному.

Краткое доказательство. $\Rightarrow$ Пусть f вычисляется ДМТ с рабочей лентой размера $O(\log n)$.

Чтобы проверить, $|f(x)| \geq l$, можно запустить моделирование f на x , одновременно подсчитывая длину выхода, пока не будет напечатано l символов. Если моделирование закончилось раньше, ответ — нет. Память — $O(\log n)$. Чтобы проверить $(f(x))_i = 1$, моделируем $f(x)$, пропуская первые $i - 1$ символов выхода, затем читаем следующий символ и сравниваем с 1. Всё делается на рабочей ленте $O(\log n)$ (храним текущую позицию выхода i , который тоже $O(\log n)$ бит).

$\Leftarrow$ Имеем оракулы для определения длины и битов выхода. Для вычисления $f(x)$ последовательно определяем длину $L = |f(x)|$ (двоичным поиском, используя первый язык). Затем для $i = 1..L$ определяем бит $(f(x))_i$ с помощью второго языка и выводим его. Вся работа занимает $O(\log n)$ памяти (храним L и текущий i). $\square$

Замечание. PATH является NL-полной задачей относительно сводимости $\leq_l$

6.2 Сертификатное определение NL

Определение 6.6. $A \in \text{NL}$, если $\exists V : \forall x(x \in A \Leftrightarrow \exists y V(x, y) = 1)$, причем V работает на логарифмической памяти по $|x|$ и читает y слева направо (двигаться влево нельзя).

Утверждение 6.2. Сертификатное определение NL эквивалентно исходному.

Доказательство. $\Rightarrow$ если мы имеем НМТ, разрешающую A , то каждый бит сертификата — индекс ветвления

$\Leftarrow$ если мы имеем V , то НМТ будет печатать все возможные сертификаты и запускаем машину (можно выводить по одному биту, чтобы тратить константную память) и запускать V .

□

Теорема 6.2 (Иммермана-Селепченьи). $\text{NL} = \text{coNL}$

Доказательство. Мы уже знаем, что $\text{PATH} — \text{NL}$ -полный, но тогда $\overline{\text{PATH}} — \text{coNL}$ -полный. Тогда достаточно будет доказать, что $\overline{\text{PATH}} \in \text{NL}$. Заметим, что $\overline{\text{PATH}} = \{(G, s, t) : \text{в графе } G \text{ нет пути из } s \text{ в } t\}$.

Пусть мы знаем C — количество вершин, достижимых из s . Тогда доказательство, что t не достижимо из s состоит из трех частей:

1. Список всех достижимых вершин из s
2. Доказательства достижимости для каждой из вершин списка
3. Вершины в списке должны быть разные (для этого пишем вершины в порядке возрастания индексов)

Первых двух пунктов легко добиться:

$$s = \boxed{v_1 \mid \text{путь } s \rightsquigarrow v_1 \mid v_2 \mid \text{путь } s \rightsquigarrow v_2 \mid \dots \mid v_C \mid \text{путь } s \rightsquigarrow v_C}$$

Осталось понять, как узнать C . Пусть C_i — число вершин, достижимые из s за $\leq i$ шагов. Известно, что $C_0 = 1, C = C_n$. Заметим, что, имея C_i , проверить утверждение "из s в v существует путь длины $\leq i + 1$ " можно аналогично предыдущему случаю. Пусть $\{v_1, \dots, v_{C_i}\}$ — вершины, достижимые за $\leq i$ шагов. Тогда пути не существует тогда и только тогда, когда $v \neq v_j$ и $(v, v_j) \notin E$ для любого j .

Далее по индукции восстанавливаем C_i : C_0 нам уже известно, покажем, как по C_i определить C_{i+1} . Все просто: для каждой вершины пишем либо сертификат достижимости, либо недостижимости.

Таким образом, сертификат будет выглядеть следующим образом:

$$s = \boxed{\text{сертификат } C_1, C_0 \mid \text{сертификат } C_2, C_1 \mid \dots \mid \text{сертификат } C_n, C_{n-1}}$$

□

Следствие (Общая теорема). Из доказательства теоремы следует, что $s(n) \geq \log n \Rightarrow \text{NSPACE}(s(n)) = \text{coNSPACE}(s(n))$

6.3 NL-полные задачи

Определение 6.7. $SCONNECTED = \{G \mid G \text{ сильно связан}\}$.

Замечание. $SCONNECTED \in NL$. Сертификатом будет являться перебор всевозможных путей (i, j) , упорядоченных лексикографически.

Можно также в качестве сертификата взять цикл, проходящий по всем вершинам.

Утверждение 6.3. $PATN \leq_l SCONNECTED$

Доказательство. Пусть дан граф G и две вершины s, t . Допишем в него все ребра вида $v \rightarrow s, t \rightarrow v$ по всем $v \in V$. Тогда полученный граф G' будет сильно связным $\Leftrightarrow$ из s в t был путь в начальном графе. Очевидно, что это можно сделать на логарифмической памяти. $\square$

Определение 6.8. $2SAT = \{\varphi \mid \varphi \text{ — выполнимая 2-КНФ}\}$.

Заметим, что по 2-КНФ можно построить следующий граф: сначала все дизъюнкты перепишем при помощи импликаций: $a \vee b \Leftrightarrow \neg a \rightarrow b \Leftrightarrow \neg b \rightarrow a$. Заметим, что если есть путь $p \rightarrow \neg p \Rightarrow p = 0$ и $\neg p \rightarrow p \Rightarrow p = 1$. Это подводит нас к следующей теореме:

Теорема 6.3. $\varphi \text{ — выполнима} \Leftrightarrow$ ни для одной вершины нет одновременно путей $p \rightarrow \neg p$ и $\neg p \rightarrow p$

Упражнение. Доказать теорему (было на алгосах).

Следствие. Таким образом, мы доказали, что $2SAT \leq_l PATN$. В качестве сертификата достаточно взять доказательство отсутствия одного из путей $p \rightarrow \neg p$ или $\neg p \rightarrow p$.

Утверждение 6.4. $\overline{PATN} \leq_l 2SAT$.

Доказательство. Рассмотрим следующую формулу: $s \wedge \neg t \wedge \bigwedge_{(v_i, v_j) \in E} (v_i \rightarrow v_j)$ в 2-КНФ. Заметим, что любой выполняющий набор φ удовлетворяет условию: $\exists s \rightarrow v \Rightarrow v = 1$. Тогда выполняющий набор существует $\Leftrightarrow t$ недостижимо из s . $\square$

Следствие. $2SAT$ — NL-полная.

7 Схемная сложность

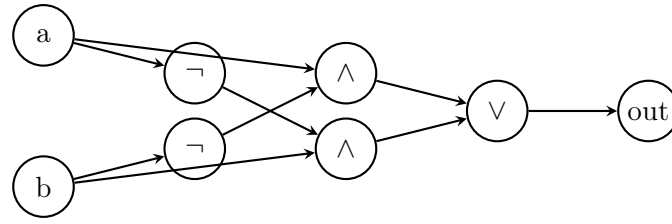
Пусть дан ориентированный граф без циклов с метками на вершинах $\{in, out, \neg, \wedge, \vee\}$, причем у вершины $\deg_{out} = 1$ для всех вершин, кроме out , у которой $\deg_{out} = 0$. $\deg_{in}$ может быть равно количеству аргументов, которые можно подать в данную функцию (для $\wedge, \vee$ — произвольное).

7.1 Сложность по размеру

Выше приведена схема для $a \oplus b$.

Пусть у нас есть язык $A \subset \{0, 1\}^*$. Положим $A^n = A \cap \{0, 1\}^n$, такие множества можно взаимно однозначно отобразить в функции $f_n : \{0, 1\}^n \rightarrow \{0, 1\}$ (0 — нет, 1 — да).

Определение 7.1. Будем говорить, что $A \in \mathbf{SIZE}(s(n))$, если $\exists \{C_n\}_{n=1}^\infty : \forall x (x \in A \Leftrightarrow C_{|x|}(x) = 1)$ и число вершин в логической схеме $C_n = O(s(n))$.



Замечание. Заметим, что $\{C_n\}_{n=1}^\infty$ — бесконечная последовательность, поэтому даже неразрешимые языки можно распознать схемами

Замечание. $\text{ALL} = \text{SIZE}(2^n \cdot n)$ — достаточно записать КНФ или ДНФ для функции f_n

Теорема 7.1 (Лупанов). $\text{ALL} = \text{SIZE}(2^n/n)$

Теорема 7.2 (Шеннон). Уменьшить оценку Лупанова нельзя

Определение 7.2. $\text{P/poly} = \text{SIZE}(\text{poly}(n))$

Рассмотрим другой подход к P/poly

Определение 7.3. $A \in \text{DTIME}(t(n))/a(n)$, если $\exists \{a_n\}_{n=1}^\infty$, M — машина тьюринга, такие, что $|a_n| = O(a(n))$, $\text{time}_M(x, a) = O(t(|x|))$ и $x \in A \Leftrightarrow M(x, a_{|x|}) = 1$.

Теорема 7.3. $\text{P/poly} = \text{DTIME}(\text{poly}(n))/\text{poly}(n)$

Доказательство. $\subset$ Положим $a_n = C_n$, M будет вычислять $C_n(x)$.

$\supset$ Аналогично доказательству в теореме Кука-Левина

□

Также существует еще один подход к P/poly : будем рассматривать только прямолинейные программы, содержащие только присваивание. Например, для $a \oplus b$:

```

⊕(a, b) {
    c = ¬ a
    d = ¬ b

    e = a ∧ d
    f = b ∧ c

    return e ∨ f
}
  
```

Теорема 7.4. $\text{P} \subsetneq \text{P/poly}$

Доказательство. Рассмотрим $K \subset \mathbb{N}$ — неразрешимое множество. Положим $A = \{x : |x| \in K\} \in \text{P/poly}$, но $\notin \text{P}$, т.к. K неразрешимо □

Вспомним, что мы изучаем вопрос $\text{P} \stackrel{?}{=} \text{NP}$. Существует следующая гипотеза:

Гипотеза: $\text{NP} \not\subseteq \text{P/poly}$.

Теорема 7.5. $\text{EXPSPACE} \not\subseteq \text{P/poly}$

Доказательство. TODO □

Теорема 7.6 (Карпа-Липтона). $\mathbf{NP} \subset \mathbf{P}/\mathbf{poly} \Rightarrow \Sigma_2^p = \Pi_2^p$

Доказательство. TODO □

Теорема 7.7. $\mathbf{EXP} \subset \mathbf{P}/\mathbf{poly} \Rightarrow \mathbf{EXP} = \Sigma_2^p \Rightarrow \mathbf{P} \neq \mathbf{NP}$

Доказательство. TODO □

Во времена, когда Шеннон придумывал эту науку, схемы моделировали реальные платы, стоимость которых зависела от их размера. Теперь посмотрим на это по другому: будем смотреть на скорость вычисления, т.е. на глубину схемы:

7.2 Сложность по времени

Определение 7.4. Глубина схемы — максимальная длина пути из входа в выход

Определение 7.5. $\mathbf{NC}^d$ — языки, распознаваемые схемами размера $\mathit{poly}(n)$ глубины $O(\mathit{poly}(n))$, состоящими из $\neg$ и $\wedge, \vee$ валентности 2.

Определение 7.6. $\mathbf{AC}^d$ — языки, распознаваемые схемами размера $\mathit{poly}(n)$ глубины $O(\log^d n)$, состоящими из $\neg$ и $\wedge, \vee$ произвольной валентностью.

Замечание. $\mathbf{NC}^d \subset \mathbf{AC}^d \subset \mathbf{NC}^{d+1}$

Следствие. $\mathbf{NC}^0 \subset \mathbf{AC}^0 \subset \mathbf{NC}^1 \subset \mathbf{AC}^1 \subset \dots$

При этом на схемы C_n можно накладывать различные ограничения. Например:

1. Никаких ограничений
2. Полиномиальная равномерность: $n \mapsto C_n$ вычисляется за $\mathit{poly}(n)$
3. Логарифмическая равномерность: $n \mapsto C_n$ вычисляется на $O(\log n)$ памяти. В этом случае $\mathbf{NC}^1 \subset \mathbf{L} \subset \mathbf{NL} \subset \mathbf{NC}^2$. **В общем случае это неверно!**

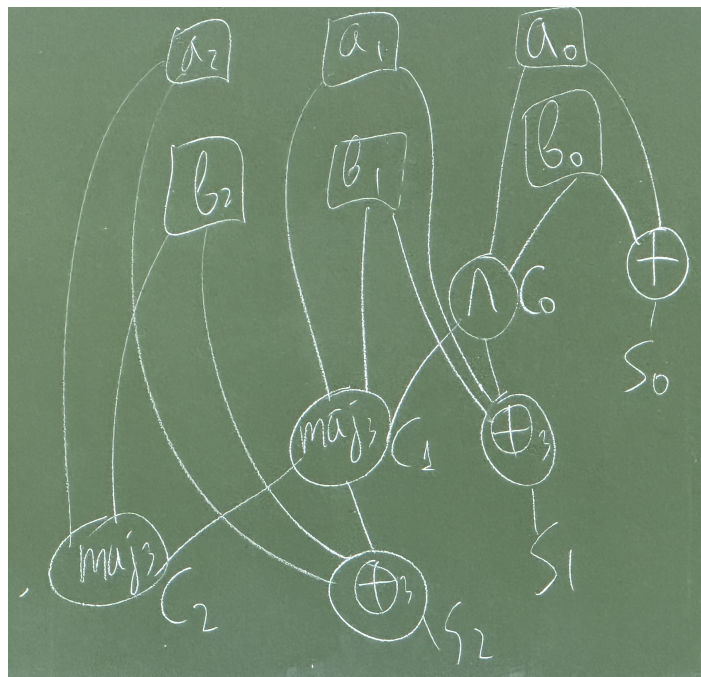
Замечание. $\mathbf{NC}^0 \subsetneq \mathbf{AC}^0$: рассмотрим задачу $\bigwedge_n$ — делает конъюнкцию всех переменных (не лежит в $\mathbf{NC}^0$, т.к. там лежат задачи с зависимостями только от $O(1)$ переменных).

Замечание. $\mathbf{AC}^0 \subsetneq \mathbf{NC}^1$: рассмотрим задачу $\bigwedge_n \oplus$ всех переменных. Лежит в $\mathbf{NC}^1$, т.к. можно рекурсивно делить пополам и делать $\oplus$ полученных частей.

Вопрос $\mathbf{NP} \stackrel{?}{\subset} \mathbf{NC}^1$ является открытой проблемой.

7.3 Схемы малой глубины для $+$, $\cdot$

Пусть $s = a + b$. Тогда схема для s :



7.4 Быстрые сумматоры

TODO

7.5 Быстрые мультипликаторы

Идея: по тройке (x, y, z) получим тройку t, v , такую, что $x + y + z = t + v$ за константное время. Тогда мы уменьшили количество слагаемых в $\frac{2}{3}$ раз. Таким образом, если нам надо посчитать $a \cdot b$, мы можем посчитать это за $\log_{\frac{3}{2}} b$ и мы свели задачу к предыдущей

8 Вероятностная сложность

Будем рассматривать машину Тьюринга $M(x, r)$ — как машину Тьюринга одного аргумента с некоторыми случайными битами r (по сути, r — бесконечная последовательность случайных бит). Будем рассматривать машины, которые работают за $\text{poly}(|x|)$

Пример. Вспомним тест проверки на простоту Миллера-Рабина. Ответы на одном входе выдавались с вероятностями:

MR	$P(MR = 1)$	$P(MR = 0)$
$n \in \mathbb{P}$	1	0
$n \notin \mathbb{P}$	1/4	3/4

Пример. Рассмотрим алгебраические схемы: они строятся аналогично логическим, но в узлах могут находиться $+$, $\cdot$. Возникает задача: по алгебраическим схемам P, Q понять, совпадают ли построенные по ним многочлены? Заметим, что $\deg P \leq 2^s$, где s — размер схемы. Заметим, что существует такой результат:

Лемма 8.1 (Шварца-Зипшеля). Пусть $S \in \mathbb{F}_p[x_1, \dots, x_n]$, $\deg S \leq d$, причем $S \not\equiv 0$. Тогда $P(S(x_1, x_2, \dots, x_n) \neq 0) \geq 1 - \frac{nd}{p}$.

Возьмем $p \gg 2^s$, но экспоненциальное от s . Посчитаем $P(x), Q(x)$ для случайного x . Если $P(x) \neq Q(x) \Rightarrow P \neq Q$. В противном случае скорее всего $P = Q \pmod p$. Будем брать p случайно, поэтому, скорее всего $P \neq Q \Rightarrow P \neq Q \pmod p$.

8.1 Вероятностные классы

Вероятностные классы можно определять двумя числами: $P(M(x, r) = 1 | x \in A)$ и $P(M(x, r) = 1 | x \notin A)$. Запишем известные нам вероятностные классы и добавим еще другие:

Класс	$P(M(x, r) = 1 x \in A)$	$P(M(x, r) = 1 x \notin A)$
P	1	0
NP	> 0	0
coNP	1	< 1
RP	$> 1/2$	0
coRP	1	$< 1/2$
BPP	$> 2/3$	$< 1/3$
PP	$\geq 1/2$	$< 1/2$
ZPP	1	0

Замечание. В **ZPP**, требование на время работы следующее: оно должно быть полиномиально в среднем, в смысле матожидания (а не просто полиномиально).

Мы докажем некоторые вложения (остальные на семинары/самостоятельное изучение)

Замечание. $\mathbf{RP} \subset \mathbf{NP}$, $\mathbf{coRP} \subset \mathbf{coNP}$, $\mathbf{BPP} \subset \mathbf{PP}$, $\mathbf{P} \subset \mathbf{ZPP}$ следуют непосредственно из определений.

Утверждение 8.1. $\mathbf{ZPP} \subset \mathbf{RP}$

Доказательство. $\mathbb{E}time_M(x, r) = t(|x|) \Rightarrow P(time_M \geq 2t) < \frac{1}{2}$ (неравенство Маркова). Соответственно, запускаем $M(x, r)$ на $2t(|x|)$ шагов. Если машина остановилась, даем тот же ответ, иначе — 0. $\square$

Утверждение 8.2. $\mathbf{RP} \subset \mathbf{BPP}$

Доказательство. Используем прием Амплификации. Запустим $M(x, r)$ два раза: $M(x, r_1) \vee M(x, r_2)$, где $r_1 r_2 = r$. Тогда вероятность получить 1 будет $\geq 3/4$. $\square$

Замечание (Амплификация). Для **RP** вместо $> \frac{1}{2}$ можно взять $> \alpha$ для любого $\alpha \in (0, 1)$ и даже функции $\alpha(n) \in \left(\frac{1}{poly(n)}, 1 - \frac{1}{exp(n)}\right)$. Рассмотрим $M_k(x, r_1, \dots, r_k) = M(x, r_1) \vee \dots \vee M(x, r_k)$. Тогда заметим, что $P(M_k = 0) = (1 - \alpha(n))^k$. Будем запускать машину $p(n)q(n)$ раз, тогда $P(M_k = 0) = (1 - \alpha(n))^k \approx \frac{1}{e^{q(n)}}$.

Замечание. **BPP**: Рассмотрим $M_k(x, r_1, \dots, r_k) = Maj(M(x, r_1), \dots, M(x, r_k))$, где Maj — выдает 1, если хотя бы половина равна 1. Эта конструкция позволяет подставить в **BPP** любое число.

Теорема 8.1 (Эйдлмана). $\mathbf{BPP} \subset \mathbf{P}/poly$

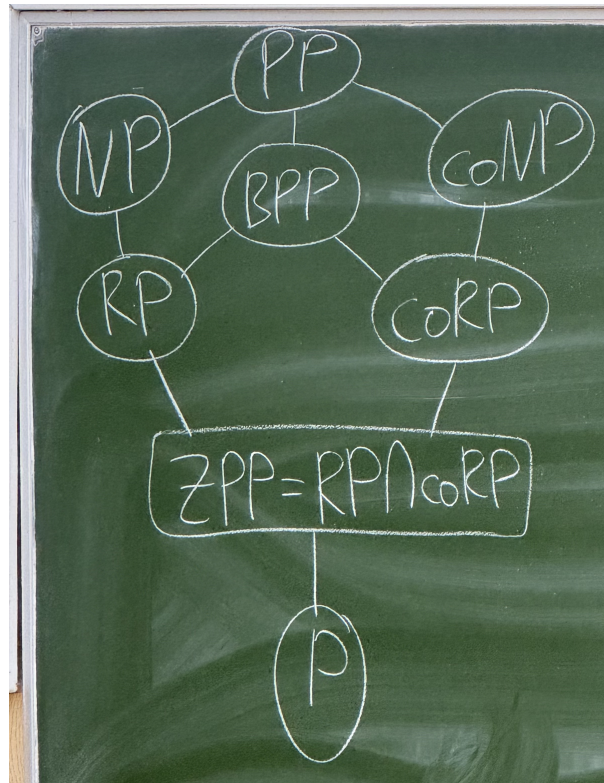


Рис. 1: Взаимоотношения вероятностных классов между собой (включение идет снизу вверх)

Доказательство. TODO

□

Теорема 8.2 (Гача-Сипсера-Лаутемана). $BPP \subset \Sigma_2^P \cap \Pi_2^P$

Доказательство. TODO

□

Следствие. $P = NP \Rightarrow P = BPP$

Теорема 8.3 (Hardness vs. Randomness). "Если NP сильно сложнее P ", то $P = BPP$.

Идея. Уменьшить количество используемых случайных битов до $O(\log n)$, дальше запускаем полный перебор.

□

8.2 Метод условных матожиданий

TODO

9 Интерактивные доказательства

Традиционное понимание доказательство — статический текст, который проверяется некоторым верификатором (вывод из аксиом и тд). Сейчас мы ознакомимся с другим подходом к доказательству, а именно, интерактивному доказательству:

В доказательстве участвуют две стороны: Prover и Verifier. Они имеют диалог, после которого Verifier выносит вердикт (0 или 1).

Опишем данную процедуру формально. Изначально есть некоторое множество A и $x \in A$, который известен и прувверу и верификатору. Верификатор хочет отвечать "1" тогда и только тогда, когда $x \in A$. Прувер, в свою очередь, хочет, чтобы верификатор всегда отвечал "1".

Кроме того, на верификатора накладывается ограничение: он должен работать за $O(poly(|x|))$.

Замечание. Любой язык из **NP** можно разрешить на полученной модели вычисления

Доказательство. Одна итерация диалога реализует сертификатное доказательство **NP** □

Чтобы отделить данный класс языков от **NP**, необходимо добавить в верификатор случайные биты.

Пример. Допустим, что пруввер отличает цвета монет (красный и зеленый), а верификатор — нет (и его цель — это проверить). Разрешается это следующим образом: верификатор спрашивает, где находится какой цвет, после чего подбрасывает (скрытно от пруввера) монетку. После чего, в зависимости от результата броски, меняет монеты или нет и повторяет вопрос. Так происходит достаточно долго.

В итоге, если пруввер действительно различает цвета, то он должен угадывать каждый раз, когда мы его спрашиваем, и нет иначе (но есть вероятность, что он их не различает и просто угадывает).

Итак, опишем формально формат диалога:

$$\begin{aligned} m_1 &= P(x) \\ m_2 &= V(x, r, m_1) \\ m_3 &= P(x, r, m_1, m_2) \\ &\vdots \\ m_{2k+1} &= P(x, r, m_1, m_2, \dots, m_{2k}) \\ a &= V(x, r, m_1, m_2, \dots, m_{2k+1}) \in \{0, 1\} = V^P(x) \end{aligned}$$

Определение 9.1. $A \in \mathbf{IP}$, если $\exists V$, т.ч.:

1. $x \in A \Rightarrow \exists P : \Pr\{V^P(x) = 1\} \geq \frac{2}{3}$
2. $x \notin A \Rightarrow \exists P : \Pr\{V^P(x) = 1\} \leq \frac{1}{3}$

Замечание. $\mathbf{NP} \subset \mathbf{IP}$, $\mathbf{BPP} \subset \mathbf{IP}$.

Пример (Неизоморфность графа). Рассмотрим задачу $\mathbf{GNI} = \{(G_0, G_1) : G_0 \not\cong G_1\}$. Будем выбирать случайное $b \in \{0, 1\}$ и $\sigma \in S_n$. Пошлем $\sigma(G_b)$ прувверу и будем ждать от него число c (по сути, случайно перемешаем граф и спросим, какой это граф из исходных). Заметим, что в случае изоморфных графов, вероятность угадать будет равна $\frac{1}{2}$, поэтому за два

Теорема 9.1. $\mathbf{IP} \subset \mathbf{PSPACE}$

Доказательство. Фиксируем x . Пусть протокол состоит из $k = \text{poly}(|x|)$ раундов, верификатор использует r длины $\text{poly}(|x|)$. Определим

$$W_i(m_1, \dots, m_i) = \max_P \Pr[V^P \text{ принимает} \mid \text{диалог } m_1, \dots, m_i].$$

Рекурсивно:

- ▷ Если следующий ход за верификатором: $W_i = \mathbb{E}_r[W_{i+1}]$.
- ▷ Если за доказывающим: $W_i = \max_{m_{i+1}} W_{i+1}$.

Вычисляем W_0 рекурсивно, храня стек глубины $O(k)$ и текущий диалог. Всё это требует $O(\text{poly}(|x|))$ памяти. Затем сравниваем W_0 с $2/3$ и $1/3$. Следовательно, $A \in \mathbf{PSPACE}$ $\square$

Определение 9.2 (Класс $\mathbf{MA}$). Язык $L \in \mathbf{MA}$, если $\exists$ вероятностная машина Тьюринга V (верификатор) и полином p , такие что:

1. **Полнота:** $\forall x \in L \exists y \in \{0,1\}^{p(|x|)} : \Pr_r[V(x,y,r) = 1] \geq \frac{2}{3}$.
2. **Корректность:** $\forall x \notin L \forall y \in \{0,1\}^{p(|x|)} : \Pr_r[V(x,y,r) = 1] \leq \frac{1}{3}$.

Здесь r — случайная строка длины $p(|x|)$, используемая верификатором.

Определение 9.3 (Класс $\mathbf{AM}$). Язык $L \in \mathbf{AM}$, если $\exists$ вероятностная машина Тьюринга V и полином p , такие что:

1. **Полнота:** $\forall x \in L \exists$ стратегия Merlin: $\Pr_r[V(x, r, m(r)) = 1] \geq \frac{2}{3}$.
2. **Корректность:** $\forall x \notin L \forall$ стратегий Merlin*: $\Pr_r[V(x, r, m^*(r)) = 1] \leq \frac{1}{3}$.

Здесь r — случайная строка Arthur (длины $p(|x|)$), m — ответ Merlin (длины $p(|x|)$), зависящий от r .

Теорема 9.2. $\mathbf{MA} \subseteq \mathbf{AM}$.

Доказательство. Пусть $L \in \mathbf{MA}$ с верификатором $V(x,y,r)$. Построим протокол $\mathbf{AM}$:

1. Arthur выбирает случайную строку r (той же длины, что и в V) и отправляет её Merlin.
2. Merlin, получив r , выбирает $y = y(r)$ и отправляет его Arthur.
3. Arthur вычисляет $V(x, y, r)$ и выдаёт результат.

Полнота. Если $x \in L$, то по определению $\mathbf{MA}$ существует фиксированный сертификат y_0 такой, что

$$\Pr_r[V(x, y_0, r) = 1] \geq \frac{2}{3}.$$

Стратегия Merlin: независимо от r отправлять y_0 . Тогда Arthur принимает с вероятностью $\geq 2/3$.

Корректность. Пусть $x \notin L$. Для любой стратегии Merlin $y(r)$ имеем:

$$\Pr_r[V(x, y(r), r) = 1] \leq \max_y \Pr_r[V(x, y, r) = 1] \leq \frac{1}{3},$$

поскольку для каждого фиксированного y вероятность принятия $\leq 1/3$ (корректность **МА**). Следовательно, условие корректности **АМ** выполнено.

Таким образом, $L \in \mathbf{АМ}$. □

Замечание. Обратное включение $\mathbf{АМ} \subseteq \mathbf{МА}$ неизвестно и считается маловероятным (тогда бы коллапсировала полиномиальная иерархия).